

# Blending and Texture

## Discussion #4

Written by:

Nithin

---

## Topics

This section covers blending and textures.

## Logistics

Remember Project 2 is due Friday, 9/29 at 11:59pm!

## 1 Taking it apart, and putting it together

We want to separate an image into details at different scales before blending it with another image. Let  $I$  be a  $256 \times 256$  grayscale image. You may use the following operations:

- $\text{blur}(X)$ : apply a normalized Gaussian low-pass filter.
- $\text{down}_2(X)$ : keep every other row and column.
- $\text{expand}_2(X)$ : upsample and interpolate to twice the height and width.

You may ignore boundary effects and numerical rounding.

(a) Set  $G_0 = I$ . Write expressions for the next two Gaussian pyramid levels,  $G_1$  and  $G_2$ , and give their sizes. Does the order of blurring and downsampling matter?

(b) Construct Laplacian detail levels  $L_0$  and  $L_1$  from these Gaussian levels. Store  $L_2 = G_2$  as the coarsest level. Write expressions for  $L_0$  and  $L_1$ . What does each of  $L_0, L_1, L_2$  represent and what are their sizes?

(c) Starting with  $L_2$ , write the steps to recover  $I$ . Does the downsampling prevent exact reconstruction when all three levels are available? Why or why not?

(d) Suppose Himanshu discards  $L_2$  because it “just looks blurry.” Can he still recover the original image from  $L_0$  and  $L_1$  alone?

## 2 Hiding the seam

Two different images  $A$  and  $B$  contain both broad, smooth color variations and fine texture. We want to blend the two images such that the left side comes from  $A$  and the right side from  $B$ .

Let  $M$  be a mask that is 1 on the left half and 0 on the right half. Ordinary feathering replaces this hard boundary with a smoothed mask  $\tilde{M}$  transition:

$$C = \tilde{M} \odot A + (1 - \tilde{M}) \odot B,$$

where  $\odot$  denotes pixel-wise multiplication.

(a) Consider two choices for the transition: a very narrow one and a very wide one. Which is more likely to leave an abrupt color seam? How about ghosted textures? Are there cases where we wouldn't want to use a wider transition?

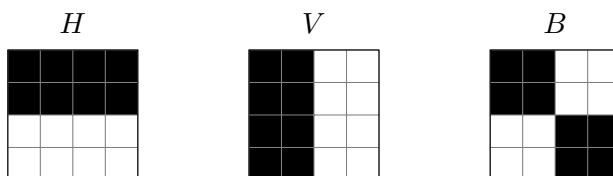
(b) Suppose we separate each image into high frequency bands and a coarse, low-frequency image. Which should use a narrower transition and which should use a wider transition? Give your answer in the original image's coordinates.

(c) Let  $L_k^A$  and  $L_k^B$  be the levels constructed in Problem 1. We also build a Gaussian pyramid  $M_k$  of the mask, with  $M_0 = M$ . Write an expression for the combined level  $L_k^C$ . How can we obtain the final image?

(d) Why did we use Gaussian levels for the mask rather than Laplacian levels of the mask?

### 3 Same histogram, same texture?

$H$ ,  $V$ , and  $B$ , are all images where each square is one pixel: black is 0 and white is 1.



(a) For each image, compute its normalized pixel histogram  $[p(0), p(1)]$ . Can this histogram distinguish the three patterns? What information is lost?

(b) Instead, we can measure the differences between neighboring pixels:

$$D_x I[r, c] = I[r, c + 1] - I[r, c], \quad D_y I[r, c] = I[r + 1, c] - I[r, c].$$

Ignoring boundary effects (no wrapping; no padding) and *without* computing every entry, which patterns have nonzero  $D_x$  responses? Which have nonzero  $D_y$  responses?

(c) Summarize each image with two numbers:

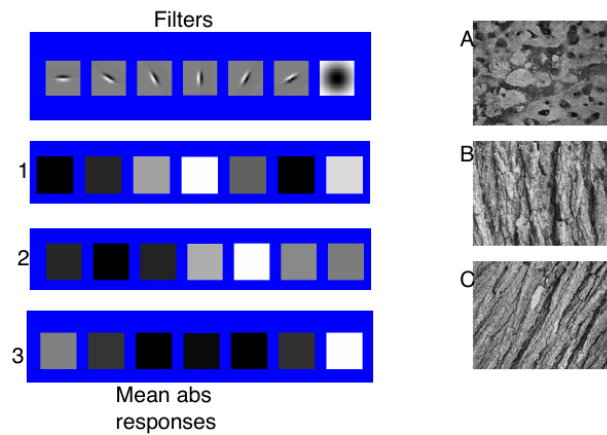
$$f(I) = [\text{mean}(|D_x I|), \text{mean}(|D_y I|)].$$

Each difference array should have 12 entries. Ignoring boundary effects, compute  $f(H)$ ,  $f(V)$ , and  $f(B)$ . Why can this descriptor distinguish patterns that the pixel histogram cannot? Give example patterns that  $f$  can not distinguish between.

## 4 Filter responses as texture descriptors

Recall this lecture slide:

Can you match the texture to its histogram?



Each descriptor contains the *mean absolute response* to each filter.

(a) (*Warmup*) Match each texture to its descriptor.

(b) Why summarize the *absolute* filter responses rather than their signed means?

(c) If we rotate a texture by  $90^\circ$ , should we expect its descriptor to remain unchanged? What descriptor can we use that would be more robust to rotations?